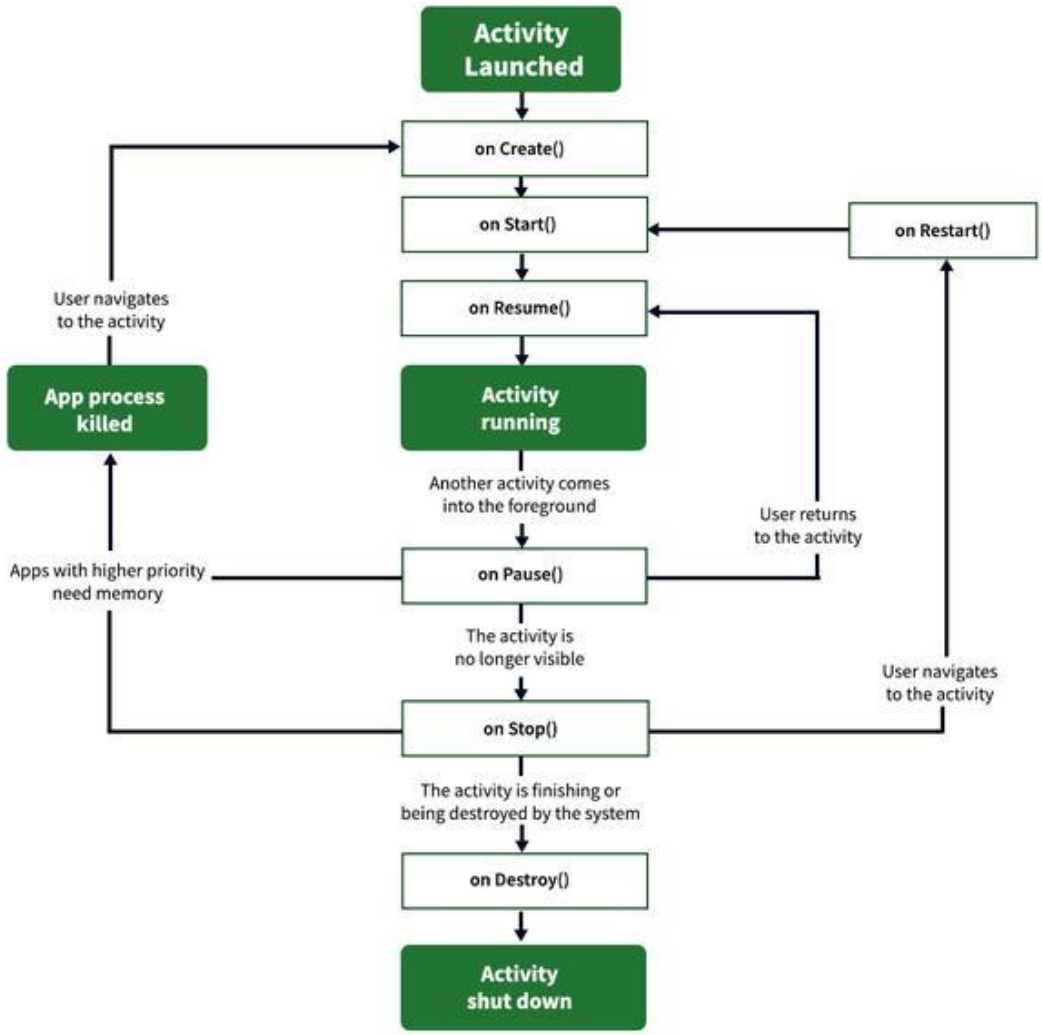
# ANDROID

# 1. Android life cycle with neat diagram

Android Activity Life Cycle is the **process that shows how an activity (screen)** starts, works, and ends in an Android app.
It helps the system **manage memory and user interaction efficiently**.

An activity does not stay in one state. It **moves through different stages** depending on user actions like opening, closing, or switching apps.

**Stages of Activity Life Cycle**




Activity Lifecycle in Android

1. **onCreate()**
   - First method called when activity starts
   - Used to **create the activity**
   - Set layout using setContentView()
   - Initialize variables, buttons, etc.
2. **onStart()**
   - Activity becomes **visible on screen**
   - Not fully ready for interaction
3. **onRestart()**
   - Called when activity is **coming back after being stopped**
   - Followed by → onStart() → onResume()
4. **onResume()**
   - Activity comes to **foreground**
   - User can **interact** (click, type)
   - This is the **active/running state**
5. **onPause()**
   - Activity is **partially hidden**
   - Another activity (like call or dialog) comes in front
   - Used to **pause tasks and save data**
6. **onStop()**
   - Activity is **completely hidden**
   - Not visible but still in memory
   - System may remove it if memory is needed
7. **onDestroy()**
   - Activity is **destroyed permanently**
   - Called when user closes the app or system removes it
   - Used to **release resources**

## 2. Fundamental components of android

Android applications are built using **basic building blocks called components**.
Each component has a **specific role** and they work together to make the app function properly.

**Main Components**

**1. Activity**

- Represents a **single screen** in an app
- It is the **main component for user interaction**
- Example: Login screen, Home screen
- Each app can have multiple activities

**2. Service**

- Runs in the **background without user interface**
- Used for **long-running tasks**
- Example: Playing music, downloading files, background syncing

**3. Broadcast Receiver**

- Used to **receive system or app messages (broadcasts)**
- It reacts when a specific event happens
- Example: Battery low, incoming call, Wi-Fi connected

**4. Content Provider**

- Used to **store and share data between apps**
- Provides **secure access** to data
- Example: Sharing contacts, images, or files

**5. Intent (Supporting Component)**

- Used to **connect different components**
- Helps in **starting activities or services**
- Example: Opening another screen or app

**Simple Diagram**

```
      Intent
        ↓
Activity ↔ Service
  ↓          ↓
Broadcast  Content
Receiver   Provider
```

## 3. JSON and its data type

JSON is a **simple and lightweight data format** used to **store and exchange data**.
In Android, JSON is mainly used to **communicate with servers (APIs)** to send and receive data.

It is based on **key–value pairs**, where each key has a value.

**Features of JSON**

- Easy to **read and understand**
- Easy to **write**
- **Language independent** (works with Java, Kotlin, Python, etc.)
- **Lightweight** (uses less memory)
- Supports **nested data** (objects inside objects)

**Basic Structure**

- Data is written inside **{ } (curly braces)**
- Uses **key : value format**
- Keys are always in **double quotes**

**Example**

```
{
  "name": "Lulu",
  "age": 20,
  "isStudent": true,
  "marks": [80, 85, 90]
}
```

**JSON Data Types**

1. **String**
   - Text value inside quotes
   - Example: "name": "Lulu"
2. **Number**
   - Integer or decimal values
   - Example: "age": 20
3. **Boolean**

- true or false
- Example: "isStudent": true

4. **Array**
- Collection of values inside **[ ]**
- Example: "marks": [80, 85, 90]

5. **Object**
- Group of key–value pairs inside **{ }**
- Example: "address": {"city": "Palakkad", "pin": 678001}

6. **Null**
- Represents no value
- Example: "middleName": null

**Uses of JSON in Android**

- Sending data to server
- Receiving data from APIs
- Storing structured data

**Advantages**

- Simple and easy to learn
- Less data size
- Fast data transfer

## 4. SQLite

SQLite is a **lightweight database** used in Android to **store data locally** on the device.
It is **built into Android**, so no separate installation is needed.

It uses **SQL (Structured Query Language)** to store, retrieve, update, and delete data.

**Features of SQLite**

- **Serverless** (no separate server needed)
- **Lightweight and fast**
- Works **offline (no internet required)**
- Uses **less memory**

- Supports **standard SQL commands**

**Main Components of SQLite**

1. **SQLiteOpenHelper**
   - Helps to **create and manage database**
2. **SQLiteDatabase**
   - Used to **perform database operations** (insert, update, delete)
3. **ContentValues**
   - Used to **store values** while inserting data
4. **Cursor**
   - Used to **read data** from database

**SQLite Data Types**

- **INTEGER** → Whole numbers
- **TEXT** → String values
- **REAL** → Decimal numbers
- **BLOB** → Binary data (images, files)
- **NULL** → Empty value

**Uses of SQLite**

- Storing user data
- Offline apps (notes, contacts, etc.)
- Small to medium data storage

**Advantages**

- No internet needed
- Easy to use
- Fast performance
- Secure local storage

**Limitations**

- Not suitable for very large data
- Limited multi-user support

## 5. JVM and DVM

| Feature | JVM (Java Virtual Machine) | DVM (Dalvik Virtual Machine) |
|---|---|---|
| File Type | Runs **.class (bytecode)** | Runs **.dex files** |
| Platform | Used in **Java (desktop, web, server)** | Used in **Android OS** |
| Architecture | **Stack-based** | **Register-based** |
| Memory Usage | **High memory consumption** | **Low memory consumption** |
| Performance | Slower for mobile devices | **Faster and optimized for mobile** |
| Power Usage | More battery usage | **Battery efficient** |
| Execution | Directly executes bytecode | Converts .class → .dex |
| Environment | General-purpose computing | Mobile environment |
| Instance | One JVM can run multiple programs | Each app runs in **separate VM** |

## 6. Indent in android

Indent in Android means the **space (gap)** given around a view or text to make the layout **clean and readable**.

It helps in **proper alignment and better UI design**.

**Types of Indent in Android**

**1. Indent in Code (Formatting)**

- It means adding **spaces or tabs** in XML or Java/Kotlin code
- Used to make code **easy to read and understand**

**Example:**

```
<LinearLayout>
   <TextView
```

```
    android:text="Hello"/>
</LinearLayout>
```

Here, TextView is indented inside LinearLayout

**2. Indent in UI (Design)**

***a) Padding (Inside Indent)***

- Space **inside the view**, between content and border
- Example:

```
android:padding="16dp"
```

***b) Margin (Outside Indent)***

- Space **outside the view**, between other views
- Example:

```
android:layout_margin="16dp”
```